*Universidade Federal de Viçosa – Campus Florestal*
*Instituto de Ciências Exatas e Tecnológicas*
*Bacharelado em Ciência da Computação*

# Trabalho Prático 2
# Algoritmos e Estruturas de Dados 1

**Professora:** Thais Regina de Moura Braga Silva
**Valor:** 10 pontos
**Data da Entrega (slides + código):** 29/10/2026
**Data das Entrevistas:** 03/10/2026 e 05/10/2026
**Forma de Entrega:** Moodle (formato .zip ou .tar.gz)



O objetivo deste trabalho prático é permitir a avaliação do impacto causado pelo desempenho dos algoritmos em sua execução real. Vimos em sala de aula que existem problemas e algoritmos de complexidade exponencial, chamados de intratáveis. Nesses casos, os programas, ao serem executados, podem demorar uma quantidade de tempo não razoável para encontrar uma solução, dependendo do tamanho da entrada. Vamos observar, portanto, como isso ocorre na prática.

Como vocês sabem, capturar um Pokémon muitas vezes exige batalhar com ele. Por isso, antes de iniciar a missão de captura dos Pokémon fugitivos apresentada no Trabalho Prático 1 (TP1), vocês deverão decidir quais Pokémon levarão para formar sua equipe de batalha. Além disso, o centro de pesquisa precisa monitorar o desempenho dos Pokémon escolhidos para compor essa equipe. Para isso, cada um deles é equipado com um PokeMonitor, que monitora seu status durante as batalhas. No entanto, existe um problema: esse equipamento de monitoramento consome energia, e a Agência de Experimentos e Desenvolvimento Silph (AEDS) dispõe de uma quantidade limitada de energia para manter esses equipamentos funcionando durante as missões. Dessa forma, os pesquisadores encontraram um dilema: quanto mais forte um Pokémon é, maior é sua contribuição para o desempenho da missão, porém, mais energia é necessária para que seu PokeMonitor funcione corretamente. Portanto, o objetivo será selecionar os Pokémon que maximizem a força total da equipe, sem ultrapassar a capacidade máxima de energia disponível para os Pokemonitores.

Esse problema será denominado **Problema de Seleção de Pokémon**. Para resolvê-lo neste trabalho, será utilizada uma estratégia de força bruta, na qual todas as combinações possíveis de Pokémon serão avaliadas para identificar a melhor equipe. Sendo a melhor equipe o conjunto de ambos os treinadores. Dessa forma, o programa deverá ser executado para diferentes tamanhos de entrada N, e o tempo gasto para encontrar a solução deverá ser medido, permitindo observar o impacto do crescimento da entrada no desempenho do algoritmo.

Utilizando os N Pokémon fornecidos como entrada, o programa deverá encontrar o subconjunto que maximize a soma das forças dos Pokémon selecionados, respeitando a capacidade máxima de energia disponível para manter os PokeMonitores funcionando. Cada Pokémon poderá ser selecionado no máximo uma vez, e a equipe poderá conter qualquer quantidade de cada Pokémon, inclusive nenhum, desde que respeite a restrição de energia.

A entrada, semelhante ao utilizado no trabalho anterior, conterá os nomes dos treinadores, a quantidade de Pokémon disponíveis (N) para seleção e também o limite máximo de energia disponível para os PokeMonitores da equipe. Em seguida, serão fornecidas N linhas com as informações de cada Pokémon, contendo seu número na Pokédex, nome, tipo, sua pontuação de poder e a quantidade de energia consumida por seu PokeMonitor.

**Atenção:** a implementação deve ser feita para este trabalho de maneira independente da feita para o TP1, ou seja, você deve iniciar um novo projeto na sua IDE para este trabalho prático. Entretanto, é esperado que você utilize dos TADs criados para o TP1, ou seja, o TAD Pokémon, TAD Pokelista, TAD Treinador.

Existem diversas possíveis implementações para o "Problema de Seleção de Pokémon". Entretanto, as mais utilizadas requerem recursos de programação que ainda não foram estudados por vocês. Dessa forma, adotaremos uma estratégia mais direta, visto que o objetivo principal do trabalho é a avaliação do desempenho, e não o desenvolvimento do algoritmo. Cada grupo deverá implementar um programa, em linguagem C, da seguinte forma:

- Procurar por um algoritmo de combinação simples X a X para um conjunto de N elementos (combinação = subconjunto sem repetição, a ordem não importa). Esse algoritmo deve, portanto, produzir todas as possíveis combinações de tamanho X dentre os N elementos. Você deverá utilizar a linguagem C. Portanto, se o algoritmo encontrado estiver em outra linguagem de programação, vocês deverão convertê-lo para a linguagem C.
- **Atenção**: se necessário, segue um link de referência sobre os conceitos de Análise Combinatória. Veja que, neste trabalho, se pede especificamente uma implementação de combinação. https://descomplica.com.br/blog/arranjo-combinacao-e-permutacoes/.
- Você deverá usar seu algoritmo de combinação para criar combinações de tamanho 1 até N de times de Pokémon, isto é, você vai criar todas as combinações de tamanho 1, depois todas de tamanho 2, em seguida todas de tamanho 3, e assim sucessivamente, até que chegue à todas as combinações de tamanho N (no caso, apenas 1). O processo de seleção deve seguir estritamente o critério de prioridade definido: deve-se buscar as combinações que possuam maior valor de pontuação de poder que respeitem o limite de energia.
- Execute o programa para os seguintes valores de N: 10, 15, 20, 30. Ao executar o programa, utilize uma biblioteca para medição do tempo de execução, como o comando `time` do Unix.

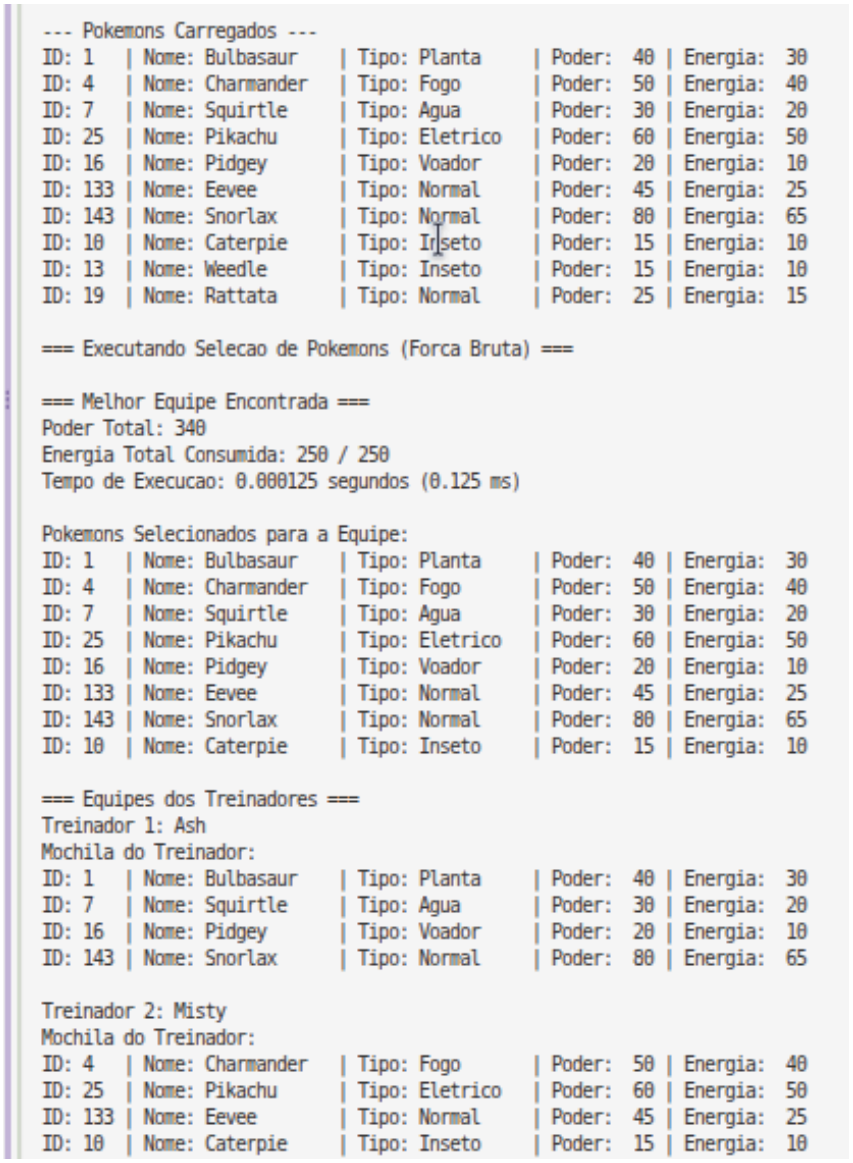
**Arquivo de Entrada**
Para o modo de utilização por arquivo, o seu programa irá ler um arquivo de entrada com o seguinte formato:

```
Ash                         // Nome do primeiro treinador
Misty                       // Nome do segundo treinador
10                          // Quantidade de Pokémon disponíveis (N)
250                         // Energia total disponível
1 Bulbasaur Planta 40 30
4 Charmander Fogo 50 40
7 Squirtle Agua 30 20
25 Pikachu Eletrico 60 50      //Infos de cada um dos N Pokémon
16 Pidgey Voador 20 10
133 Eevee Normal 45 25
143 Snorlax Normal 80 65
10 Caterpie Inseto 15 10
13 Weedle Inseto 15 10
19 Rattata Normal 25 15
```

*As informações dos Pokémon consistem de: Número na Pokédex, nome do Pokémon, tipo do Pokémon, pontuação de poder e energia gasta por seu PokeMonitor.*

**Testes**
Será disponibilizado junto com a especificação deste trabalho dois pequenos arquivos de teste. O grupo deve, no entanto, criar seus próprios casos de teste. Novos testes por arquivo serão utilizados no dia da entrevista, a ser realizada como parte da avaliação deste trabalho.

Como resolução do primeiro arquivo de teste, a saída no terminal deverá exibir as informações da melhor equipe encontrada, incluindo: o seu poder total, a energia total consumida e o tempo gasto para a execução do programa. Além disso, deve listar quais são os Pokémon que compõem a melhor equipe e os seus atributos. Por fim, deverá ser exibido a distribuição arbitrária desses Pokémon entre as PokeListas dos dois treinadores. A seguir está uma sugestão de resultado para o programa:



```
--- Pokemons Carregados ---
ID: 1   | Nome: Bulbasaur    | Tipo: Planta     | Poder:  40 | Energia:  30
ID: 4   | Nome: Charmander   | Tipo: Fogo       | Poder:  50 | Energia:  40
ID: 7   | Nome: Squirtle     | Tipo: Agua       | Poder:  30 | Energia:  20
ID: 25  | Nome: Pikachu      | Tipo: Eletrico   | Poder:  60 | Energia:  50
ID: 16  | Nome: Pidgey       | Tipo: Voador     | Poder:  20 | Energia:  10
ID: 133 | Nome: Eevee        | Tipo: Normal     | Poder:  45 | Energia:  25
ID: 143 | Nome: Snorlax      | Tipo: Normal     | Poder:  80 | Energia:  65
ID: 10  | Nome: Caterpie     | Tipo: Inseto     | Poder:  15 | Energia:  10
ID: 13  | Nome: Weedle       | Tipo: Inseto     | Poder:  15 | Energia:  10
ID: 19  | Nome: Rattata      | Tipo: Normal     | Poder:  25 | Energia:  15

=== Executando Selecao de Pokemons (Forca Bruta) ===

=== Melhor Equipe Encontrada ===
Poder Total: 340
Energia Total Consumida: 250 / 250
Tempo de Execucao: 0.000125 segundos (0.125 ms)

Pokemons Selecionados para a Equipe:
ID: 1   | Nome: Bulbasaur    | Tipo: Planta     | Poder:  40 | Energia:  30
ID: 4   | Nome: Charmander   | Tipo: Fogo       | Poder:  50 | Energia:  40
ID: 7   | Nome: Squirtle     | Tipo: Agua       | Poder:  30 | Energia:  20
ID: 25  | Nome: Pikachu      | Tipo: Eletrico   | Poder:  60 | Energia:  50
ID: 16  | Nome: Pidgey       | Tipo: Voador     | Poder:  20 | Energia:  10
ID: 133 | Nome: Eevee        | Tipo: Normal     | Poder:  45 | Energia:  25
ID: 143 | Nome: Snorlax      | Tipo: Normal     | Poder:  80 | Energia:  65
ID: 10  | Nome: Caterpie     | Tipo: Inseto     | Poder:  15 | Energia:  10

=== Equipes dos Treinadores ===
Treinador 1: Ash
Mochila do Treinador:
ID: 1   | Nome: Bulbasaur    | Tipo: Planta     | Poder:  40 | Energia:  30
ID: 7   | Nome: Squirtle     | Tipo: Agua       | Poder:  30 | Energia:  20
ID: 16  | Nome: Pidgey       | Tipo: Voador     | Poder:  20 | Energia:  10
ID: 143 | Nome: Snorlax      | Tipo: Normal     | Poder:  80 | Energia:  65

Treinador 2: Misty
Mochila do Treinador:
ID: 4   | Nome: Charmander   | Tipo: Fogo       | Poder:  50 | Energia:  40
ID: 25  | Nome: Pikachu      | Tipo: Eletrico   | Poder:  60 | Energia:  50
ID: 133 | Nome: Eevee        | Tipo: Normal     | Poder:  45 | Energia:  25
ID: 10  | Nome: Caterpie     | Tipo: Inseto     | Poder:  15 | Energia:  10
```

**Informações Importantes**

Em particular, atente para:

- O programa deve ser organizado em módulos, conforme estudado em sala de aula. O módulo do programa principal deve estar separado dos módulos que compõem os TADs.
- O programa deve estar bem indentado e comentado.
- No Makefile deve conter um comentário explicando como compilar o programa, tanto em Windows quanto em Linux.
- Caso apareçam números fixos no código, estes devem ser definidos como constantes.
- Trabalhos copiados serão penalizados.

**Atenção:** Faça uma apresentação final contendo uma breve explicação do código implementado (indicando como funciona o algoritmo de combinação utilizado, de onde

foi obtido, como o seu programa prioriza as combinações de maior pontuação de poder dos pokemón. A apresentação também deverá conter as configurações de hardware e software da máquina que foi utilizada e os resultados dos tempos de execução para os valores de N indicados acima. É obrigatório o uso do template de apresentação, conforme foi feito para o TP1.

**Responda também na apresentação à seguinte pergunta:** seria razoável executar o seu algoritmo para valores de N>=50? Justifique a resposta.

**Aviso:** Soluções que não correspondam ao que foi pedido na especificação serão duramente penalizadas por não atenderem à documentação. A especificação pode conter erros ou problemas não intencionais que podem e devem ser reportados, caso encontrados. Além disso, diversas decisões de projeto e desenvolvimento precisam e devem ser feitas por cada dupla. Implementações extras são possíveis e serão bem-vindas, desde que tudo o que foi explicitamente pedido nesta especificação tenha sido contemplado, e que o objetivo central do tema do trabalho não seja comprometido.

**O que entregar?** Deverá ser entregue um arquivo no formato .zip, nomeado no formato "TP1Aluno1Matricula1Aluno2Matricula2", contendo: (1) todo código-fonte produzido, incluindo os arquivos de cabeçalho; e (2) um conjunto de slides, utilizando o formato disponibilizado no Moodle. O formato para a slides é pdf.

**Como será a avaliação?** O trabalho será avaliado através de uma apresentação/entrevista com as pessoas monitoras e a avaliação do código pela professora e pelas pessoas monitoras.